НЕКОММЕРЧЕСКОЕ АКЦИОНЕРНОЕ ОБЩЕСТВО «КАЗАХСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ имени К.И.САТПАЕВА»

Институт Автоматики и информационных технологий
Кафедра Программной инженерии

CSE5032 Разработка веб-сервисов
Задание 2 Модуль 5-8

Зав. кафедры ПИ **Абдолдина Ф. Н.**
Экзаменатор **Герцен Е.А.**

Модуль 5-8

Цель работы
Модернизировать разработанный ранее University Course Management API, обеспечив безопасный доступ к данным, управление пользователями и ролями, аутентификацию и авторизацию, а также поддержку нескольких версий API.

Важное условие: **все функциональные возможности первой части задания должны продолжать работать**. Повторная разработка CRUD, Repository, DTO, AutoMapper и других ранее реализованных компонентов не требуется.

1. Настройка использования Web API
Необходимо подготовить существующий API для взаимодействия с внешними клиентскими приложениями.

Требования:
1. Настроить CORS с использованием именованной политики, разрешающей запросы от двух конкретных адресов клиентских приложений.
2. Реализовать дополнительный ASP.NET Core клиент (небольшое MVC- или Razor Pages-приложение), взаимодействующий с University Course Management API.
3. Для выполнения HTTP-запросов использовать IHttpClientFactory.
4. Клиент должен позволять получать список курсов и сведения о конкретном курсе.
5. Предусмотреть корректную обработку недоступности API, ошибок соединения и ответов с неуспешными HTTP-кодами.
6. Настроить конфигурацию адреса API через appsettings.json, без жёсткого указания адреса в программном коде.

Результат: отдельное ASP.NET Core приложение, которое получает данные через HTTP, а не обращается непосредственно к базе данных University Course Management API.

2. Защита доступа к API
Необходимо разделить существующие endpoints на публичные и защищённые.

Требования:
1. Определить, какие операции доступны анонимным пользователям, а какие требуют аутентификации.
2. Использовать атрибуты [Authorize] и [AllowAnonymous].
3. Запретить анонимное создание, изменение и удаление студентов, преподавателей, курсов и записей на курсы.
4. Настроить возврат 401 Unauthorized при отсутствии или недействительности аутентификации.
5. Настроить возврат 403 Forbidden, когда пользователь аутентифицирован, но не обладает необходимыми правами.
6. Исключить передачу конфиденциальных данных пользователей в ответах API.

Вы должны самостоятельно объяснить разницу между аутентификацией и авторизацией и продемонстрировать её на примере своего приложения.

3. Реализация JWT-аутентификации
Необходимо реализовать механизм выдачи и проверки JWT Access Token.

Требования:

1. Создать endpoint для входа пользователя, например POST /api/auth/login.
2. При успешной аутентификации выдавать JWT.
3. Включить в токен идентификатор пользователя, имя пользователя и его роли.
4. Настроить проверку подписи, издателя (Issuer), аудитории (Audience) и срока действия токена.
5. Установить ограниченный срок действия Access Token.
6. Реализовать передачу токена через HTTP-заголовок Authorization: Bearer.
7. Обеспечить отклонение запросов с просроченным, изменённым или некорректным токеном.
8. Предусмотреть безопасное хранение ключа подписи вне исходного кода и Git-репозитория.

Вам необходимо продемонстрировать работу защищённого endpoint с корректным JWT и без него.

4. ASP.NET Core Identity

Необходимо интегрировать ASP.NET Core Identity в существующее приложение для управления учётными записями пользователей.

Требования:

1. Настроить ASP.NET Core Identity с использованием Entity Framework Core.
2. Реализовать регистрацию пользователя.
3. Реализовать вход с проверкой пароля через Identity.
4. Использовать UserManager и SignInManager там, где это необходимо.
5. Реализовать получение информации о текущем пользователе через GET /api/auth/me.
6. Запретить регистрацию пользователей с одинаковым именем пользователя.
7. Настроить минимальные требования к сложности пароля.
8. Предусмотреть блокировку учётной записи после нескольких неудачных попыток входа.
9. Обеспечить хранение паролей в виде защищённых хешей средствами Identity, без самостоятельной реализации алгоритма хеширования.

Регистрация пользователя не должна автоматически предоставлять ему административные права.

5. Авторизация на основе ролей

Необходимо реализовать три роли пользователей.

Роль	Полномочия
Admin	Управление пользователями и ролями, полный доступ к университетским данным
Teacher	Просмотр курсов, изменение оценок студентов только по собственным курсам
Student	Просмотр доступных курсов, собственных записей и оценок

Дополнительные требования:

1. Реализовать создание и назначение ролей через ASP.NET Core Identity.
2. Настроить авторизацию с использованием ролей и политик.
3. Связать учётную запись преподавателя с существующей сущностью Teacher, а студента - с Student.

4. Преподаватель не должен иметь возможность изменять оценки по чужим курсам.
5. Студент не должен иметь возможность просматривать оценки другого студента.
6. Запретить самостоятельное повышение привилегий через API.
7. Проверять права доступа на сервере, а не только скрывать недоступные функции в клиентском приложении.

Особое внимание уделить проверке принадлежности ресурсов пользователю, поскольку одной проверки роли недостаточно.

6. Управление версиями ASP.NET Core Web API
Необходимо обеспечить одновременную поддержку двух версий API.

Требования:
1. Настроить API Versioning.
2. Реализовать версии v1 и v2 для получения информации о курсах.
3. Сохранить работоспособность контрактов первой версии.
4. Во второй версии расширить представление информации о курсах, добавив сведения о преподавателе и количестве записанных студентов.
5. Использовать отдельные DTO для изменённого контракта второй версии.
6. Настроить документацию OpenAPI для обеих версий.
7. Обеспечить возможность одновременного обращения к v1 и v2.
8. Продемонстрировать, что изменение второй версии не нарушает работу клиентов первой версии.

Пример маршрутов:
- *GET /api/v1/courses*
- *GET /api/v2/courses*
- *GET /api/v1/courses/5*
- *GET /api/v2/courses/5*

Вы самостоятельно выбираете организацию контроллеров и способ повторного использования общей бизнес-логики.

7. Комплексные сценарии безопасности
В дополнение к отдельным требованиям необходимо реализовать и проверить следующие сценарии:

№	Сценарий	Ожидаемый результат
1	Анонимный пользователь запрашивает список курсов	200 OK
2	Анонимный пользователь пытается создать курс	401 Unauthorized
3	Пользователь вводит неправильный пароль	Ошибка аутентификации без выдачи JWT
4	Пользователь входит с правильными данными	Получает JWT
5	Пользователь обращается с просроченным JWT	401 Unauthorized
6	Student пытается удалить курс	401 Unauthorized
7	Teacher изменяет оценку на собственном курсе	200 OK

8	Teacher изменяет оценку на чужом курсе	403 Forbidden
9	Student запрашивает чужие оценки	403 Forbidden
10	Admin назначает пользователю роль	Успешное изменение роли
11	Student пытается назначить себе роль Admin	403 Forbidden
12	Клиент обращается к API v1 и v2	Обе версии возвращают корректные данные

8. Дополнительные требования

В ходе модернизации приложения необходимо обеспечить:

- Сохранение ранее реализованной архитектуры и функциональности.
- Корректный порядок регистрации и выполнения Authentication и Authorization Middleware.
- Отсутствие паролей, JWT и секретных ключей в логах.
- Корректную работу Swagger/OpenAPI с JWT Bearer Authentication.
- Использование существующего механизма обработки ошибок, адаптированного для новых сценариев.
- Отсутствие дублирования бизнес-логики между версиями API.

9. Что необходимо продемонстрировать на защите

Вы должны самостоятельно запустить проект и показать работу всех новых возможностей через Swagger, Postman и разработанное клиентское приложение.

На защите преподаватель вправе выбрать любой из сценариев раздела 7 и потребовать его демонстрацию.

Дополнительно студент должен объяснить, как связаны ASP.NET Core Identity, JWT, Authentication Middleware, Authorization Middleware, Claims и Roles, а также почему проверка роли не всегда достаточна для защиты данных.

10. Что сдаёт студент

1. Ссылку на Git-репозиторий с обновлённым проектом из задания 1.
2. Рабочий University Course Management API с реализованными механизмами безопасности.
3. Дополнительное ASP.NET Core клиентское приложение.
4. Миграции базы данных и скрипты создания тестовых пользователей и ролей.
5. Swagger/OpenAPI с поддержкой JWT и версий API.
6. README с инструкцией по запуску, настройке и проверке новых функций.
7. Краткую схему прохождения запроса от клиента через аутентификацию и авторизацию до защищённого ресурса.

11. Критерии оценки - 10 баллов

Раздел	**Баллы**
Настройка Web API, CORS, HTTP-клиент	1
Защита endpoints и обработка 401/403	1
Реализация и проверка JWT	1,5
Интеграция ASP.NET Core Identity	1,5
Роли и разграничение доступа к ресурсам	2
Управление версиями API	1
Комплексные сценарии безопасности	1
Демонстрация, документация и качество интеграции	1

Итого	**10**

Итоговое требование

Задание 2 является обязательным продолжением задания 1, а не самостоятельным проектом.

Оцениваться будут прежде всего новые функциональные возможности: безопасная аутентификация, авторизация, управление учётными записями, защита данных и версионирование.

Студент, который реализует новую функциональность, но нарушит работоспособность ранее созданного API, не сможет получить максимальную оценку.

Таким образом, после выполнения обеих частей у студентов будет один законченный учебный проект - University Course Management API, последовательно развиваемый от базовой REST-архитектуры до защищённого, версионируемого Web API с управлением пользователями.